Course : AAD – ALGORITHM ANALYSIS AND DESIGN

Lecture 1

Introduction

Dr Rahaf Ghazal

2025-2026

Definition of Algorithms

- An algorithm is a step by step procedure for solving the given problem.

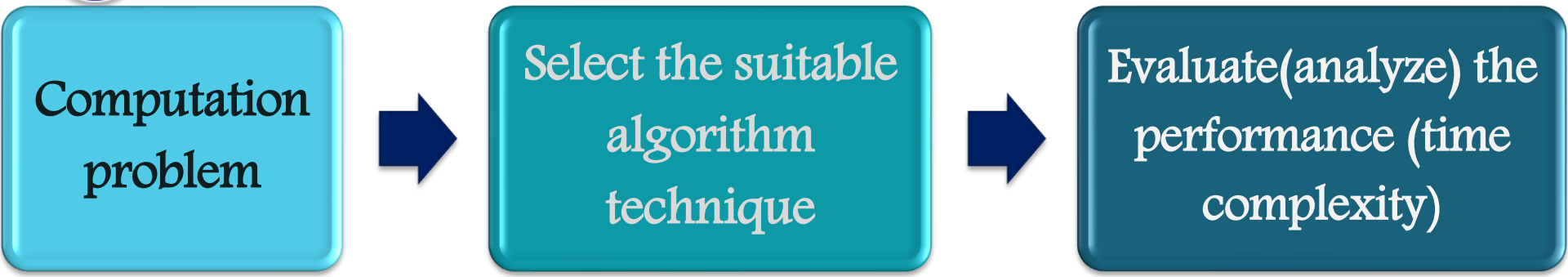
- An algorithm is independent of any programming language and machine.

Prosperities of Algorithms

- Definiteness
- Effectiveness
- Finiteness

- **Each instruction should be clear and unambiguous.**
- **It should be possible to perform each instruction using basic operations.**
- **algorithm terminates after a finite number of steps.**

The main objective of this course is to identify the different algorithm design techniques
used to solve computer problems and analyses their performance in order to select the best one.



Algorithm types

الخوارزميات التتابعية sequential algorithms : يجري تنفيذها وفق تتابع التعليمات وبترتيب معين.

الخوارزميات المتوازية parallel algorithms : يجري تنفيذ أكثر من جزء في آن واحد معاً، ويجري ذلك عادة على عدة معالجات.

الخوارزميات العودية recursive algorithms : وهي خوارزميات تستخدم ضمن تعليماتها استدعاء للخوارزمية نفسها. مثالها خوارزمية حساب !n

الخوارزميات التراجعية backtracking algorithms : وهي خوارزميات تستخدم لإيجاد حل ضمن مجموعة محاولات ممكنة، حيث تمثل المحاولات على شكل فروع في شجرة. يجري تجريب أحد الفروع، فإن لم نجد الحل نعود إلى الوراء لنختار مسارا آخرنجربه وهكذا، حتى تعثر على المسار المناسب. من أمثلتها تلوين خارطة بما لا يزيد على أربعة ألوان، تمارين المتاهات.

خوارزميات فرق تسد divide and conquer algorithms: يتم في هذا النوع من الخوارزميات أولا تقسيم المسألة المراد حلها إلى مسائل جزئية أصغر من نفس النمط ومن ثم حل تلك المسائل الجزئية بطريقة عودية. ثانيا تجميع حلول المسائل الجزئية التي تم الحصول عليها ضمن حل واحد للمسألة الأصلية. مثال على ذلك خوارزمية الفرز السريع quicksort وكذلك خوارزمية الترتيب بالدمج merge sort

خوارزميات البرمجة الديناميكية dynamic programming algorithms: عبارة عن خوارزميات تتذكر النتائج السابقة وتستخدمها لإيجاد نتائج جديدة تستخدم الخوارزميات الديناميكية عادة لإيجاد الحلول المثلى optimization problems مثال على ذلك خوارزمية Dijkstra في إيجاد أقصر مسار shortest path بين عقدتين (مدينتين).

الخوارزميات الطموحة Greedy Algorithms: تستخدم لإيجاد الحل الأمثلي للمسائل المطروحة، وهي تعمل على مراحل في كل مرحلة أولا نأخذ في لحظة معينة الحل الأمثل بدون النظر إلى النتائج (الطريقة) ، مثال عليها Huffman coding, Knapsack algorithm .

Algorithm design techniques – in our course

- الخوارزميات العودية (Recursive Algorithms)
- خوارزميات التتبع الخلفي - التراجعية (Backtracking algorithms)
- خوارزميات البرمجة الديناميكية (Dynamic Programming Algorithms)
- خوارزميات التفريق ثم الحل (Divide and Conquer Algorithms)
- الخوارزميات الجشعة والأنانية (Greedy Algorithms)

Performance Analysis

A priori Analysis (PERFORMANCE MESURMENT):

provides a theoretical understanding of an algorithm's efficiency and helps in predicting its behavior for different input sizes. **(Time Complexity)**

A posteriori Analysis (PERFORMANCE EVALUTION):

Implementing the algorithm in a machine and then calculating the time taken by the system to execute the program successfully

Runtime Measurement: "recording the actual execution time", **Scalability:** "performance changes as the dataset size", **Practical Considerations:** Factors like hardware specifications, programming language, and data storage mechanisms, etc.

In our course we care in a priori Analysis – Time complexity

Time Complexity

This is done as follows:

1. How long the algorithm takes : will be represented as a function of the size of the input. f(n)→how long it takes if ‘n’ is the size of input. **[efficiency of algorithm]**
2. How fast the function that characterizes the running time grows with the input size. “Rate of growth of running time”. The algorithm with less rate of growth of running time is considered better.**[Asymptotic analyses]**

Efficiency of algorithm

- يتعلق زمن تنفيذ خوارزمية بعدد العمليات الرئيسسة التي تنفذها الخوارزمية كتابع لحجم بيانات الدخل.

يتم حساب عدد العمليات الأساسية التي يتم تنفيذها في الخوارزمية كما يلي:

- حساب عدد العمليات لكل كتلة من التعليمات
- عند وجود تعليمات تفريع شرطي if-else نأخذ الحالة المحتملة الأسوأ وهي اختيار الفرع الذي يكون عدد عملياته أكبر.
- عند وجود حلقات فان عدد العمليات يتعلق بعدد مرات التكرار.
- عند وجود استدعاء لتوابع جزئية فان عدد العمليات هو عدد العمليات الاساسية في التابع الجزئي. وعند وجود استدعاء لتوابع جزئية عوديه فإن عدد العمليات متعلق بعدد مرات استدعاء التابع الجزئي العودي.

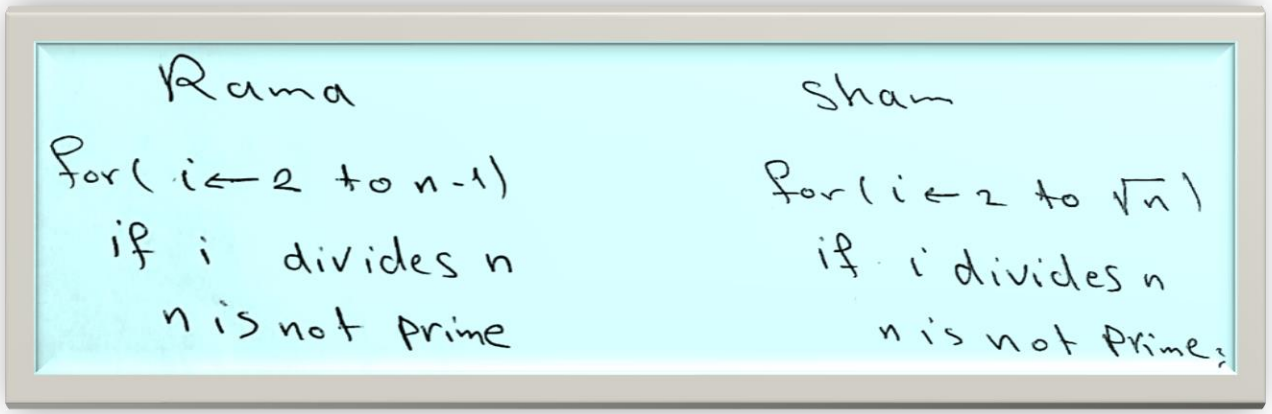
مثال تمهيدي1- دراسة كلفة خوارزمية

ALGORITHM TO CALCULATE MEAN : (*Algorithm to read a set of n numbers and calculate their mean*)	Statement	# of times executed
1. Read n.	1	1
2. Initialize Sum To 0.	2	1
3. Initialize I To 1.	3	1
4. While I<=N Do The Following	4	n+1
A. Read Number .	A	n
B. Add Number To Sum.	B	n
C. Increment I By 1.	C	n
5. Calculate Mean = Sum/N	5	1
إن كلاً من الخطوات 1, 2 , 3 ,5 تنفذ مرة واحدة بينما A,B,C ينفذ كل منها n مرة بينما 4 تنفذ n+1 مرة :	Total:	4n+5

وعليه يعطي الزمن اللازم لهذه الخوارزمية بالعلاقة **T(n)=4n+5** حيث n عدد طبيعي يمثل قيمة الدخل(عدد الأعداد في مسألتنا) ومن العلاقة نجد أن T(n) يتزايد بشكل خطي مع n. أو نقول T(n) من مرتبة n .

مثال تمهيدي2- دراسة كلفة خوارزمية

بفرض لدينا طالبتان، راما وشام، طُرح عليهما سؤال ما، وهو كتابة خوارزمية لاختبار فيما إذا كان عدد معطى أولي أم لا. فكتبت كل منهما خوارزميتيين مختلفتين (ولكنهما ستوصلان إلى الحل المطلوب) كما يلي:



نلاحظ أنه في الحالة الأسوء فإن خوارزمية راما سوف تحتاج إلى n-2 عملية قسمة للحكم على أن العدد أولي أو لا. أما شام فمن أجل نفس ال n فقد احتاجت خوارزميتها إلى $\sqrt{n} - 1$ عملية قسمة. عند تجريب قيم مختلفة لـ n سوف نلاحظ مايلي:

n	Rama	Sham
11	9 ms	2 ms
101	99 ms	9 ms
$10^6 + 3$	16.66 min	1 sec
$10^{10} + 19$	115 days	1.66 min

Analysis of Nested Loops:

```
i:=1;
While i<= n  do
Begin
    j:=1;
    while j<= m do
    begin
          j:=j+1;
    end;
    i:=i+1;
End.
```

من الملاحظ أن العدد الكلي للتكرار يساوي إلى جداء العدد الكلي لتكرار الحلقة الداخلية بالعدد الكلي لتكرار الحلقة الخارجية. وبالتالي فإن زمن التنفيذ هو:

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{m} 1 = \sum_{i=1}^{n} m = m * n$$

Example: analysis of Selection Sort Algorithm:

The following pseudocode sorts an array A[n] in ascending order :

```
procedure selectionSort(A: array)
  for i from 0 to n - 2:
      minIndex = i
      // Find the minimum element in the remaining unsorted part A[i...n-1]
      for j from i + 1 to n - 1:
            if A[j] < A[minIndex]:
                  minIndex = j
         // Swap the found minimum element with the first element of the unsorted part
      swap(A[i], A[minIndex])
```

المطلوب حساب T(n) : بفرض أن العملية الأساسية هي المقارنة:

$$T(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = n(n-1)/2$$

تحليل الحلقات- الحلقات اللوغاريتمية:

```
i:= 1;                          // executed one time
While i<250 do                  //9 comparisons
Begin
    // list of sequence statements      //c*8 operations
    i:=i*2;                             // executed  8 times
End.
```

لإيجاد $T(n)$: نلاحظ بداية أن الـ n ثابتة وقيمتها 250 .

العملية المؤثرة في الخوارزمية هي عدد مرات تنفيذ الحلقة while .

تبدأ i من القيمة واحد ويتم مضاعفتها حتى نصل إلى القيمة 250 عندها تتوقف الحلقة.

إذاً ممكن أن نحدد عدد مرات التكرار بالإجابة عن السؤال التالي: ماهي عدد المرات التي سنضرب بها 2 بنفسه حتى نصل إلى القيمة 250؟ أي نحل المعادلة التالي:

$$2^x = 250 \Rightarrow x = [Log_2(250)] = 8$$

إذا فكلفة الخوارزمية هي c*8

مثال: أحسب كلفة الخوارزمية الآتية بالنسبة لعمليات الضرب

$cin \gg n;$

$prod = 1;$

$while(n > 1)$

$n = \frac{n}{4};$

$prod = prod * n;$

$cout \ll n;$

الحل:

عند أول دخول إلى الحلقة $while$ $\Longleftarrow$ n

عند ثاني دخول إلى الحلقة $while$ $\Longleftarrow$ $\frac{n}{4}$

عند ثالث دخول إلى الحلقة $while$ $\Longleftarrow$ $\frac{n}{4^2}$

$\vdots$

عند الدخول i $\Longleftarrow$ $\frac{n}{4^{i-1}}$

عند الدخول $i+1$ $\Longleftarrow$ $\frac{n}{4^i}$

ومنه:

$$\frac{n}{4^i} = 1$$

$$\log_4 n = i$$

إذن عدد عمليات الضرب يساوي عدد التكرارات $*1$ ومنه:

$$c = 1 * \log_4 n$$

$$= \log_4 n$$

أمثلة عن تحليل حلقات مختلفة

```
for (i = 0; i < 1000; i++)
    application code
```

$$f(n) = n$$

```
for (i = 0; i < 1000; i += 2)
    application code
```

$$f(n) = n / 2$$

Multiply Loops

```
for (i = 0; i < 1000; i *= 2)
    application code
```

$$f(n) = \log n$$

Divide Loops

```
for (i = 0; i < 1000; i /= 2)
    application code
```

```
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j *= 2)
        application code
```

$$f(n) = n \log n$$

```
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
        application code
```

$$f(n) = n^2$$

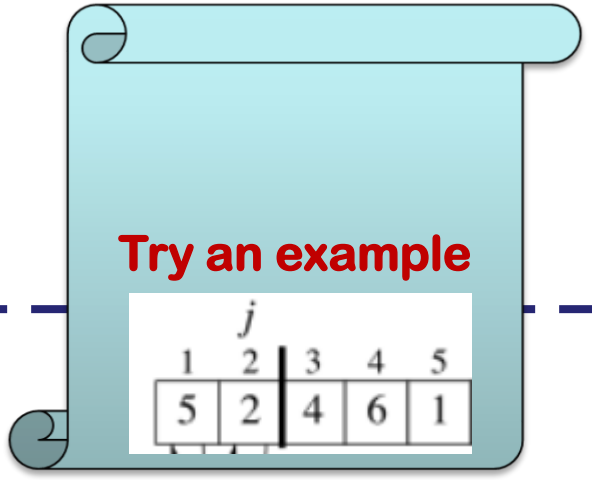
```
for (i = 0; i < 10; i++)
    for (j = 0; j < i; j++)
        application code
```

$$f(n) = n\left(\frac{n+1}{2}\right)$$

Example: analysis of Insertion Sort Algorithm:

The following pseudocode sorts an array A[n] in ascending order :

```
procedure insertionSort(A: array)
1.      for i from 1 to n - 1:
2.          key = A[i]
3.          j = i – 1
 // Move elements of A[0...i-1] that are greater than key, to one position ahead of their current position
4.          while j >= 0 and A[j] > key:
5.              A[j + 1] = A[j]
6.              j = j - 1
7.          A[j + 1] = key
end procedure
```



Let C_i be the cost of ith line.

Cost	No. Of times Executed
C_1	n
C_2	$n-1$
C_3	$n-1$
C_4	$\sum_{i=1}^{n-1}(t_i+1)$
C_5	$\sum_{i=1}^{n-1}(t_i)$
C_6	$\sum_{i=1}^{n-1}(t_i)$
C_7	$n-1$

```
1. for i from 1 to n - 1:
2.         key = A[i]
3.         j = i – 1
4.     while j >= 0 and A[j] > key:
5.             A[j + 1] = A[j]
6.             j = j - 1
7.      A[j + 1] = key
```

Running time of the algorithm is:

$$T(n) = C_1\,(n) + C_2\,(n-1) + C_3\,(n-1) + C_4\left(\sum_{i=1}^{n-1}(t_i+1)\right) + C_5\left(\sum_{j=1}^{n-1}(t_i)\right) + C_6\left(\sum_{j=1}^{n-1}(t_j)\right) + C_7\,(n-1)$$

Best case:

It occurs when Array is sorted.

All t_j values are zero.

$$T(n) = C_1\,(n) + C_2\,(n-1) + C_3\,(n-1) + C_4\left(\sum_{i=1}^{n-1}(1)\right) = C_1\,(n) + C_2\,(n-1) + C_3\,(n-1) + C_4\,(n-1) = (\boldsymbol{C_1 + C_2 + C_3 + C_4})\,\boldsymbol{n} - (\boldsymbol{C_2 + C_3 + C_4})$$

- Which is of the form **an+b**
- Linear function of n , So linear growth

Worst case:

It occurs when Array is reverse sorted.

All t_i values are "i".

$$T(n) = C_1\,(n) + C_2\,(n-1) + C_3\,(n-1) + C_4$$

$$+C_5\left(\sum_{i=1}^{n-1}(i)\right) + C_6\left(\sum_{i=1}^{n-1}(i)\right) + C_7\,(n-1)$$

$$= C_1\,(n) + C_2\,(n-1) + C_3\,(n-1) + C_4\left(\frac{n(n-1)}{2} - 1\right)$$

$$+\frac{n(n-1)}{2}(C_5 + C_6) + C_7(n-1)$$

$\Rightarrow$ T(n)= $\boldsymbol{an^2 + bn + c}$

$\Rightarrow$ **Quadratic function. So in worst case insertion sort grows in n^2**

Some of Mathematical Formula

Harmonic series

For positive integers n, the nth ***harmonic number*** is

$$\begin{aligned} H_n &= 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n} \\ &= \sum_{k=1}^{n} \frac{1}{k} \\ &= \ln n + O(1) . \end{aligned}$$

$$q + 2q + 3q + \ldots + nq = \frac{n(n+1)q}{2} \quad ; \quad 1 + 2 + 3 + \ldots + n = \frac{n(n+1)}{2}$$

$$1 + q + q^2 + \ldots + q^{n-1} = \frac{1-q^n}{1-q} \quad ; \quad 1 + q + q^2 + q^3 + \ldots + q^n = \frac{q^{n+1}-1}{q-1}$$

Sums of squares and cubes

We have the following summations of squares and cubes:

$$\sum_{k=0}^{n} k^2 = \frac{n(n+1)(2n+1)}{6} ,$$

$$\sum_{k=0}^{n} k^3 = \frac{n^2(n+1)^2}{4} .$$

Logarithms

We shall use the following notations:

$$\begin{aligned}
\lg n &= \log_2 n && \text{(binary logarithm)} ,\\
\ln n &= \log_e n && \text{(natural logarithm)} ,\\
\lg^k n &= (\lg n)^k && \text{(exponentiation)} ,\\
\lg \lg n &= \lg(\lg n) && \text{(composition)} .
\end{aligned}$$

For all real $a > 0$, $b > 0$, $c > 0$, and n,

$$\begin{aligned}
a &= b^{\log_b a} ,\\
\log_c(ab) &= \log_c a + \log_c b ,\\
\log_b a^n &= n \log_b a ,\\
\log_b a &= \frac{\log_c a}{\log_c b} ,\\
\log_b(1/a) &= -\log_b a ,\\
\log_b a &= \frac{1}{\log_a b} ,\\
a^{\log_b c} &= c^{\log_b a} ,
\end{aligned}$$

where, in each equation above, logarithm bases are not 1.

Floors & Ceilings

- For any **real** number **x**, we denote the **greatest integer** *less than or equal* to **x** by $\lfloor x \rfloor$
  - read "the floor of x"
- For any **real** number **x**, we denote the **least integer** *greater than or equal* to **x** by $\lceil x \rceil$
  - read "the ceiling of x"

وظيفة – حلل الخوارزميات التالية:

ALGO 1

```
{
int m=1,p=3,L=5;
for (int i=n; i>=1; i/=2)
for (int j=1; j<=n; j++)
{
m=m+1;                    //
p=p*2;                    //
L=L+3;                    //
}
}
```

ALGO 2

```
{
if (n==1) return;
for (int i=1; i<=n; i++)
for (int j=1; j<=n; j++)
{
cout<<"*"  ;
break;
}
}
```

أخيراً للمقارنة بين الخوارزميات لابد من تحليل تقارب توابعها الزمنية واستنتاج تابع التعقيد الزمني لكل منها

وهذا سنتعرف عليه في المحاضرة القادمة